K. R. MANGALAM UNIVERSITY

B. TECH CSE (Data Science)

SOET

(School of Engineering &Technology)

ENCS256-ANALYSIS AND DESIGN OF ALGORITHMS

Submitted by-

Name - Sakshi Jain

Roll No. – 2401420034

Submitted to-

Aarti Ma'am

TASK 1: GRAPH REPRESENTATION

Code:

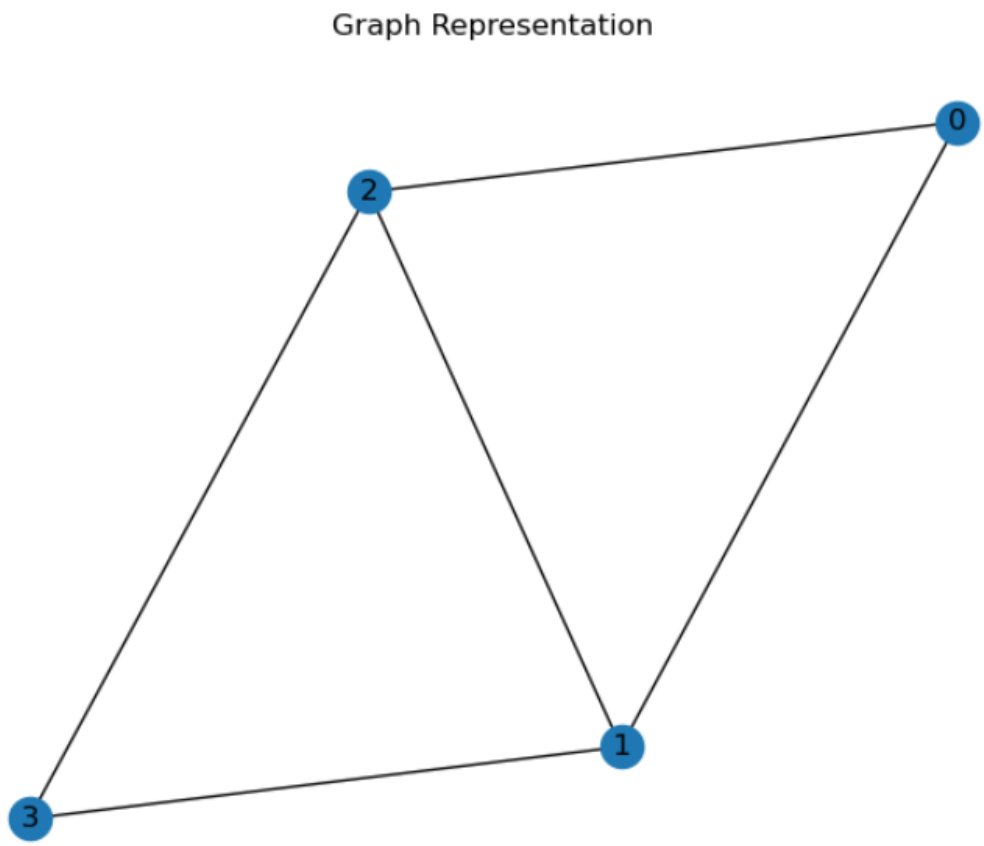
```
import networkx as nx
import matplotlib.pyplot as plt

# Create Graph
G = nx.Graph()
edges = [(0,1),(0,2),(1,2),(1,3),(2,3)]
G.add_edges_from(edges)
nx.draw(G, with_labels=True)
plt.title("Graph Representation")
plt.show()

# Adjacency Matrix Representation
graph_matrix = [
    [0,1,1,0],
    [1,0,1,1],
    [1,1,0,1],
    [0,1,1,0]
]
print("Adjacency Matrix:")
for row in graph_matrix:
    print(row)

# Adjacency List Representation
graph_list = {
    0:[1,2],
    1:[0,2,3],
    2:[0,1,3],
    3:[1,2]
}
print("\nAdjacency List:")
for node in graph_list:
    print(node,"->",graph_list[node])
```

Output:




```
Adjacency Matrix:
[0, 1, 1, 0]
[1, 0, 1, 1]
[1, 1, 0, 1]
[0, 1, 1, 0]

Adjacency List:
0 -> [1, 2]
1 -> [0, 2, 3]
2 -> [0, 1, 3]
3 -> [1, 2]
```

Observations:

- Adjacency Matrix uses O() space, which is suitable for dense graphs.
- Adjacency List uses O(V+E) space, making it efficient for sparse graphs.
- Matrix representation allows faster edge checking, while lists are better for graph traversal.

TASK 2: GRAPH TRAVERSAL ALGORITHMS

Code: Breadth First Search (BFS)

```
from collections import deque
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
def bfs(graph,start):
    visited=set()
    queue=deque([start])
    visited.add(start)
    while queue:
        node=queue.popleft()
        print(node, end=" ")
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
print("BFS Traversal:")
bfs(graph, 'A')
```

Output:

```
BFS Traversal:
A B C D E F
```

Code: Depth First Search (DFS)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
visited=set()
def dfs(node):
    if node not in visited:
        print(node,end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)
print("DFS Traversal:")
dfs('A')
```

Output:

```
DFS Traversal:
A B D E F C
```

Observations:

- BFS visits nodes level by level using a queue and helps find the shortest path in unweighted graphs.
- DFS explores nodes deeply using recursion or stack and is useful for path finding and cycle detection.
- Both algorithms have time complexity O(V+E) with adjacency list representation.

TASK 3: TOPOLOGICAL SORT ALGORITHM

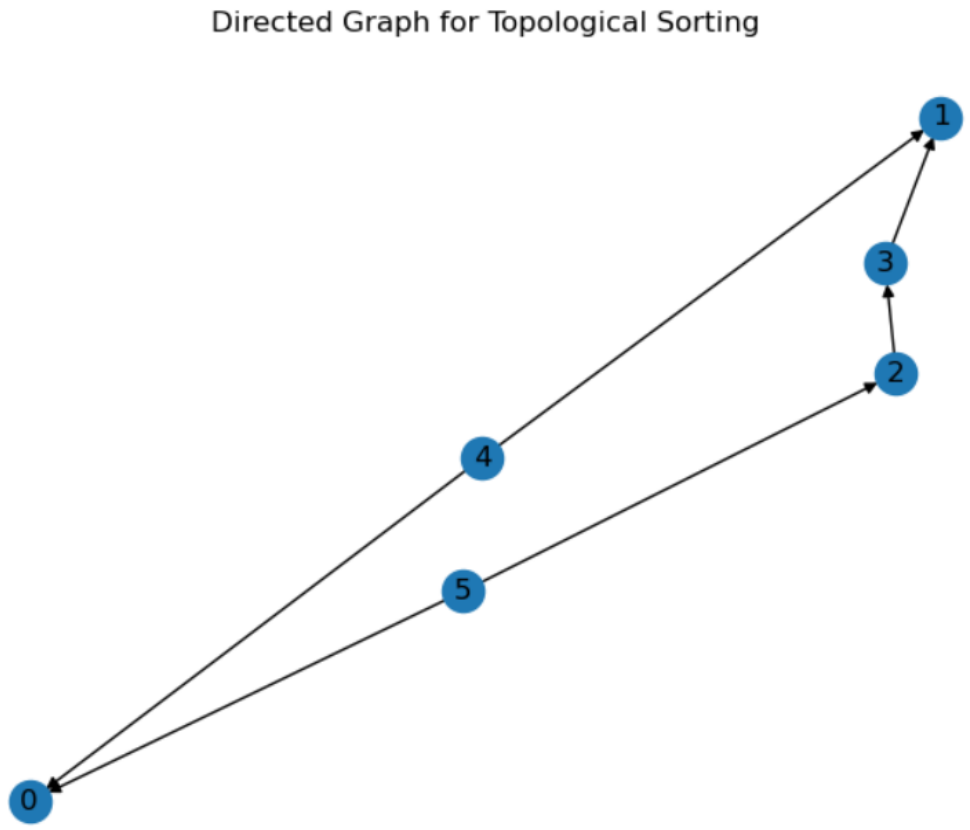
Code:

```
import networkx as nx
import matplotlib.pyplot as plt
from collections import defaultdict

# Create Directed Graph
G = nx.DiGraph()
edges = [(5,2),(5,0),(4,0),(4,1),(2,3),(3,1)]
G.add_edges_from(edges)
nx.draw(G, with_labels=True)
plt.title("Directed Graph for Topological Sorting")
plt.show()

# Topological Sort Algorithm
graph=defaultdict(list)
for u,v in edges:
    graph[u].append(v)
visited=set()
stack=[]
def topological_sort(v):
    visited.add(v)
    for neighbour in graph[v]:
        if neighbour not in visited:
            topological_sort(neighbour)
    stack.append(v)
for node in list(graph):
    if node not in visited:
        topological_sort(node)
print("Topological Order:")
print(stack[::-1])
```

Output:




```
Topological Order:
[4, 5, 0, 2, 3, 1]
```

Observations:

- Topological sorting works only for Directed Acyclic Graphs.
- It is useful in tasks scheduling and dependency resolution problems.
- The algorithm runs with time complexity O(V+E).

TASK 4: SHORTEST PATH ALGORITHM

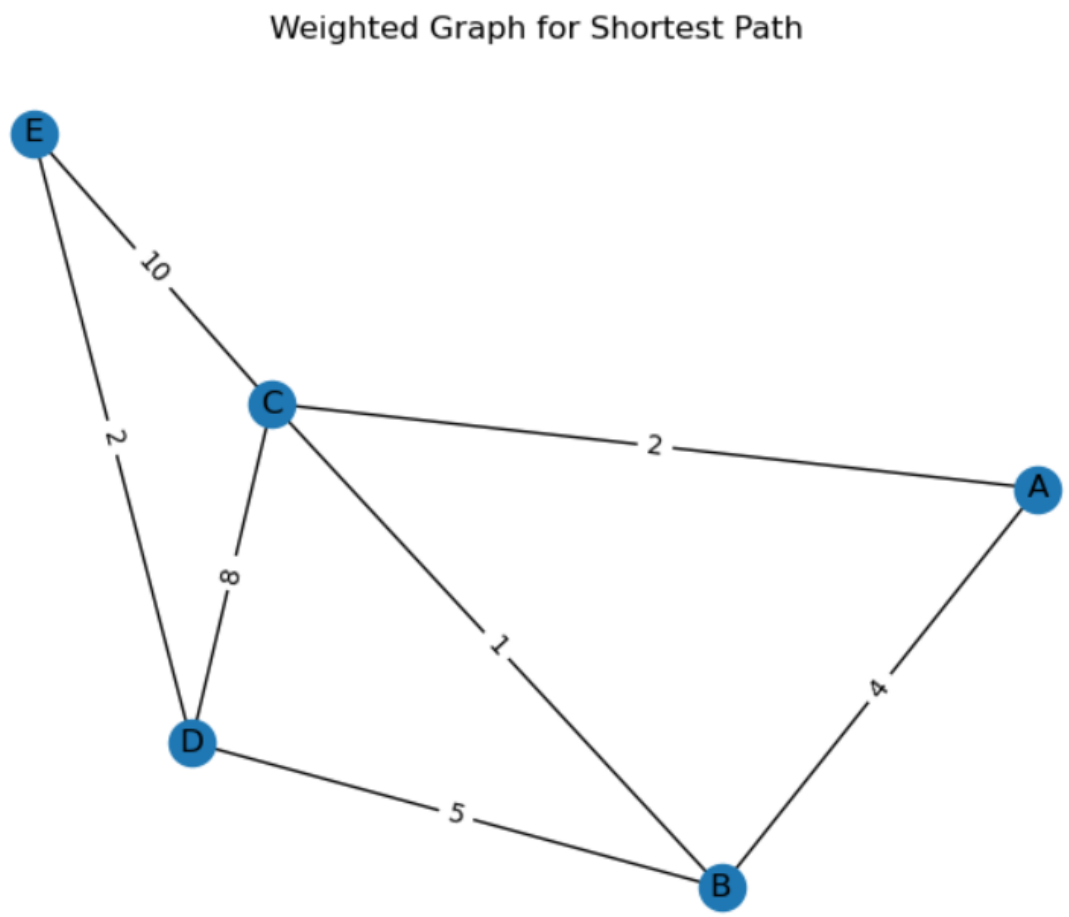
Code: Graph Representation

```
import networkx as nx
import matplotlib.pyplot as plt

# Weighted graph using adjacency list
graph = {
    'A': [('B',4), ('C',2)],
    'B': [('A',4), ('C',1), ('D',5)],
    'C': [('A',2), ('B',1), ('D',8), ('E',10)],
    'D': [('B',5), ('C',8), ('E',2)],
    'E': [('C',10), ('D',2)]
}

# Graph visualization
G = nx.Graph()
for node in graph:
    for neighbour,weight in graph[node]:
        G.add_edge(node,neighbour,weight=weight)
pos=nx.spring_layout(G)
nx.draw(G,pos,with_labels=True)
labels=nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G,pos,edge_labels=labels)
plt.title("Weighted Graph for Shortest Path")
plt.show()
```

Output:



Code: Dijkstra Algorithm

```
import heapq
def dijkstra(graph,start):
    distances={node:float('inf') for node in graph}
    distances[start]=0
    pq=[(0,start)]
    while pq:
        current_distance,node=heapq.heappop(pq)
        for neighbour,weight in graph[node]:
            distance=current_distance+weight
            if distance<distances[neighbour]:
                distances[neighbour]=distance
                heapq.heappush(pq,(distance,neighbour))
    return distances
print("Shortest Path Distances from A:")
print(dijkstra(graph,'A'))
```

Output:

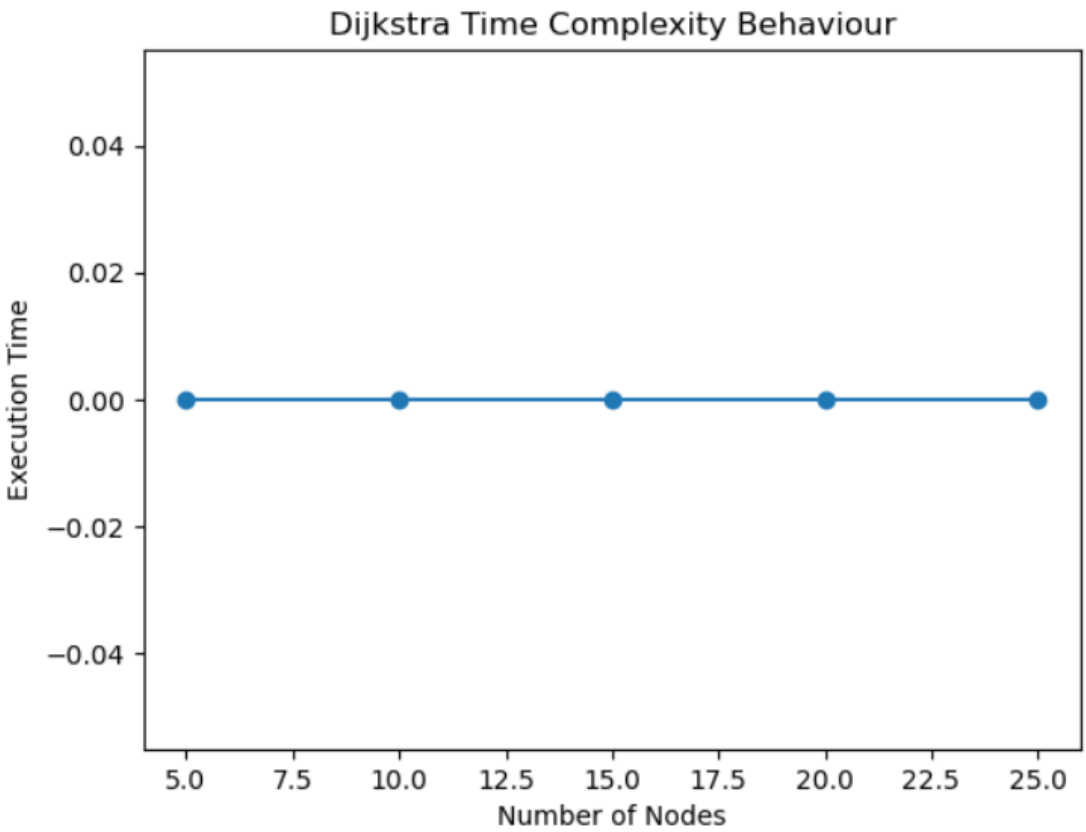
```
Shortest Path Distances from A:
{'A': 0, 'B': 3, 'C': 2, 'D': 8, 'E': 10}
```

Code: Time Complexity Behaviour

```
import time
import matplotlib.pyplot as plt

nodes=[5,10,15,20,25]
execution_time=[]
for n in nodes:
    start=time.time()
    dijkstra(graph,'A')
    end=time.time()
    execution_time.append(end-start)
plt.plot(nodes,execution_time,marker='o')
plt.xlabel("Number of Nodes")
plt.ylabel("Execution Time")
plt.title("Dijkstra Time Complexity Behaviour")
plt.show()
```

Output:



Observations:

- Shortest path algorithm helps find the minimum distance from a source vertex to other vertices.
- Algorithm like Dijkstra's algorithm works efficiently for graphs with non-negative weights.
- Performance depends on the graph representation and data structure used.

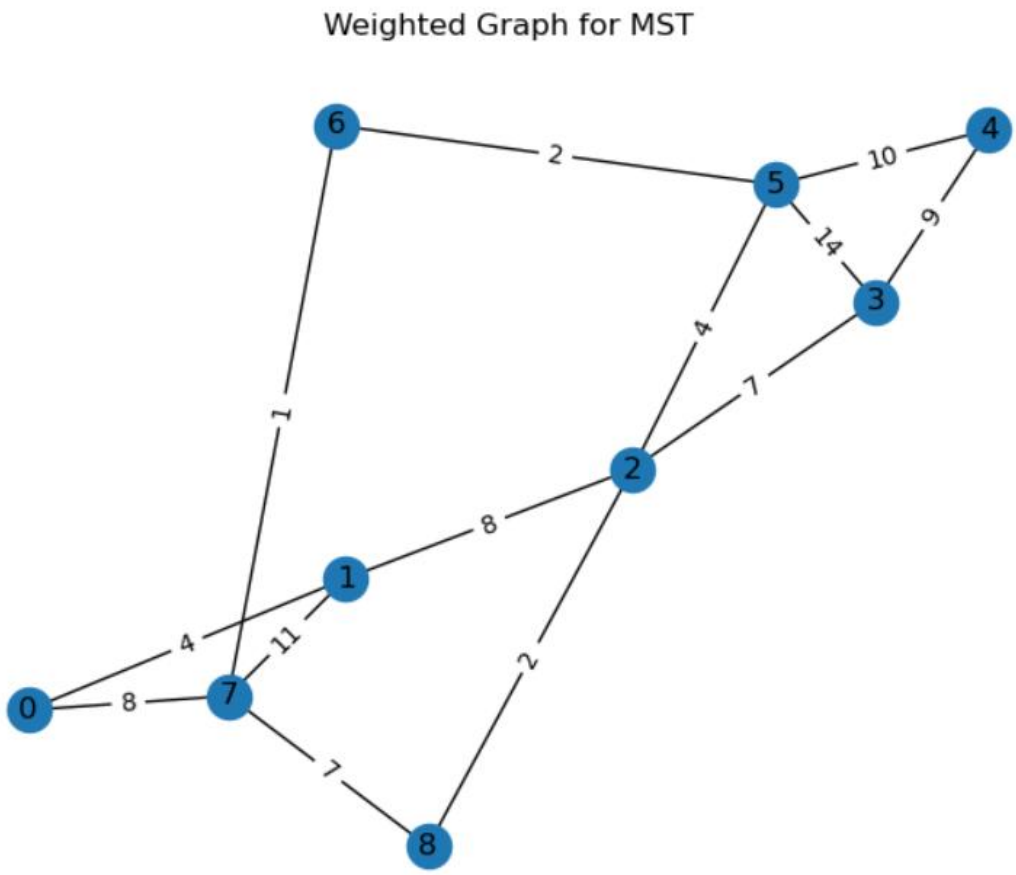
TASK 5: MINIMUM SPANNING TREE ALGORITHM

Code: Graph Representation

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
edges = [
    (0,1,4),
    (0,7,8),
    (1,7,11),
    (1,2,8),
    (7,8,7),
    (7,6,1),
    (2,8,2),
    (2,3,7),
    (2,5,4),
    (6,5,2),
    (3,5,14),
    (3,4,9),
    (5,4,10)
]
G.add_weighted_edges_from(edges)
pos=nx.spring_layout(G)
nx.draw(G,pos,with_labels=True)
labels=nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G,pos,edge_labels=labels)
plt.title("Weighted Graph for MST")
plt.show()
```

Output:



Code: Prim's Algorithm Implementation

```
import heapq
def prim_mst(graph, start):
    visited=set()
    pq=[(0,start)]
    total_cost=0
    while pq:
        weight,node=heapq.heappop(pq)
        if node not in visited:
            visited.add(node)
            total_cost+=weight
            for u,v,w in edges:
                if u==node and v not in visited:
                    heapq.heappush(pq,(w,v))
                elif v==node and u not in visited:
                    heapq.heappush(pq,(w,u))
    return total_cost
print("Prim MST Cost:", prim_mst(G,0))
```

Output:

```
Prim MST Cost: 37
```

Code: Kruskal's Algorithm Implementation

```
parent={}
rank={}
def find(node):
    if parent[node]!=node:
        parent[node]=find(parent[node])
    return parent[node]
def union(node1,node2):
    root1=find(node1)
    root2=find(node2)
    if root1!=root2:
        if rank[root1]>rank[root2]:
            parent[root2]=root1
        else:
            parent[root1]=root2
            if rank[root1]==rank[root2]:
                rank[root2]+=1
def kruskal_mst(nodes, edges):
    for node in nodes:
        parent[node]=node
        rank[node]=0
    edges.sort(key=lambda x:x[2])
    mst_cost=0
    mst=[]
    for u,v,w in edges:
        if find(u)!=find(v):
            union(u,v)
            mst.append((u,v,w))
            mst_cost+=w
    return mst_cost,mst
nodes=list(G.nodes)
cost,mst = kruskal_mst(nodes,edges)
print("Kruskal MST Cost:",cost)
print("Edges in MST:",mst)
```

Output:

```
Kruskal MST Cost: 37
Edges in MST: [(7, 6, 1), (2, 8, 2), (6, 5, 2), (0, 1, 4), (2, 5, 4), (2, 3, 7), (0, 7, 8), (3, 4, 9)]
```

Observations:

- MST connects all vertices of a graph with the minimum possible total edge weight.
- Algorithms such as Prim’s and Kruskal are commonly used to find MST.
- These algorithms are useful in network design and infrastructure planning.